

Today's agenda:

- ↳ Currency exchange
- ↳ fractional knapsack
- ↳ Greedy Properties
- ↳ Activity Selection
- ↳ Job Scheduling

Intro

↳ Subhesh

Subhesh

↳ DTU

↳ 5* on codechef

↳ ICPC → regional

↳ 3 yast teaching exp → DSA

↳ youtuber → DSA → CP → level 3
Interview sel (graphs) } Alcooling
↳ > 1M views

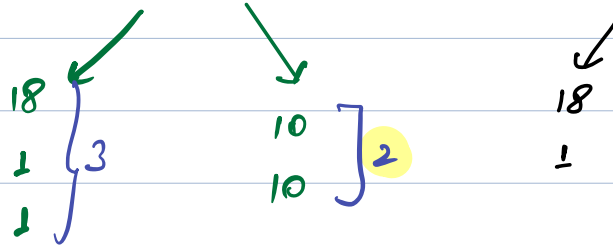
→ 1 coin always given

Indian currency: 1, 2, 5, 10, 20, 50, 100, 200, 500, 2000

Cash: 5548 → min no. of coins/notes to get reqd amount.
↳ you can repeat currencies

Notes/coins	Count	leftout amount	
2000	2	1548	
500	3	48	will it always work?
20	2	8	↓
5	1	3	NOPE
2	1	1	↳ Notes[i-1]*2 <=
1	1	0	Notes[i]

Currency: 1 10 18 amount: 20



Q) Fractional Knapsack

↳ you can consume K kg of vegetables, you can eat any integral amount of item. Max Protein you can get?

$K = 70$

vegetables	Eating complete item Protein gained	mon(Protein/kg)	eat on the basis of mon total Protein
Tomato 20 kg	200 P	$\rightarrow 10 \text{ P/kg} * 20 = 200 \text{ P}$	Onion $\rightarrow 50 \text{ kg} - 250 \text{ P}$
Apples 15 kg	180 P	$\rightarrow 12 \text{ P/kg} * 15 = 180 \text{ P}$	Tomato $\rightarrow 20 \text{ kg} \rightarrow 200 \text{ P}$
onion 50 kg	250 P	$\rightarrow 5 \text{ P/kg}$	450 P
Chicken 10 kg	150 P	$\rightarrow 15 \text{ P/kg} * 10 = 150 \text{ P}$	
Potato 25 kg	200 P	$\rightarrow 8 \text{ P/kg} * 8 = 64 \text{ P}$	
mango 12 kg	132 P	$\rightarrow 11 \text{ P/kg} * 12 = 132 \text{ P}$	
Seafood 5 kg	100 P	$\rightarrow 20 \text{ P/kg} * 5 = 100 \text{ P}$	
↑ max amount		826 P	

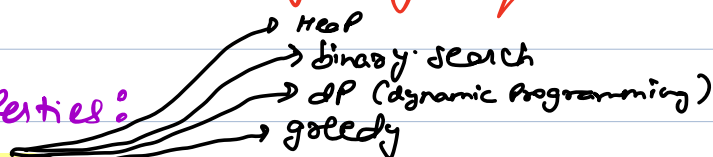
↳ Solve Problems by "locally optimal choices" at each step, and that will give you final optimal answer.

Greedy Properties:

① for min/max related problems.

② Based on what parameter we want to apply greedy.

③ either prove it logically or to cross it try to come with



Counter examples.



Why serising is important

→ donot remember things, just understand it.



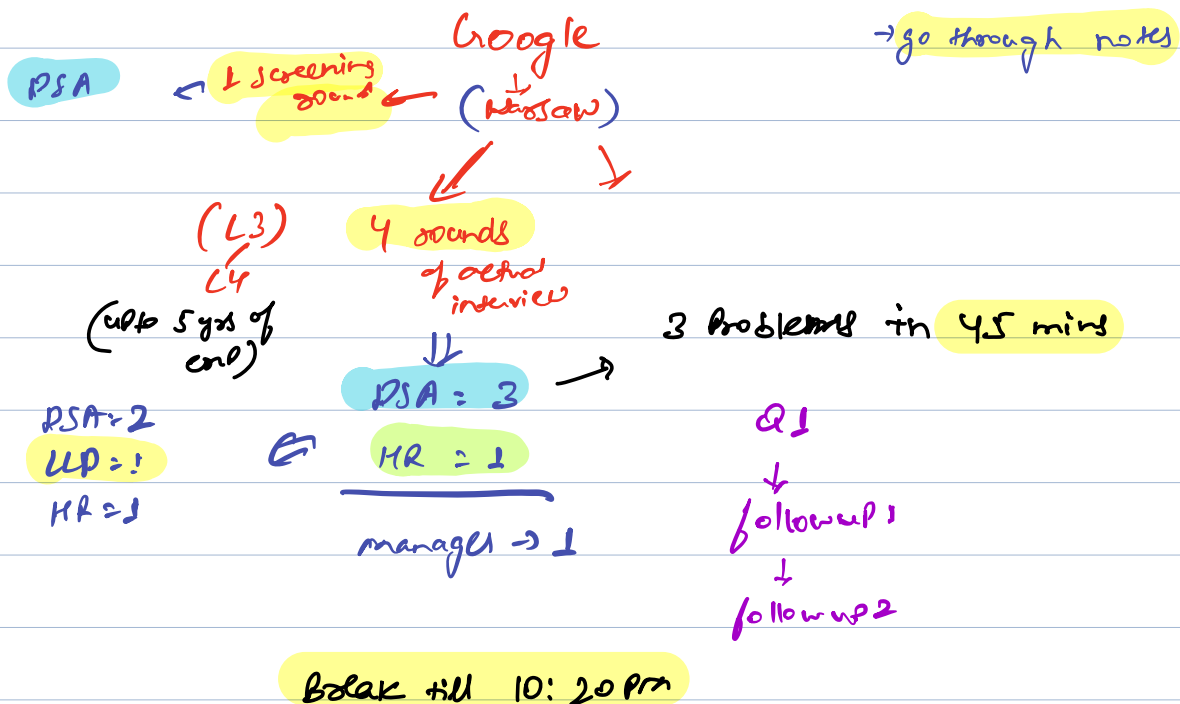
45 mins → 3 questions



Seen → extra time will be

↳ understand it → remember concept.

(revision) → every month

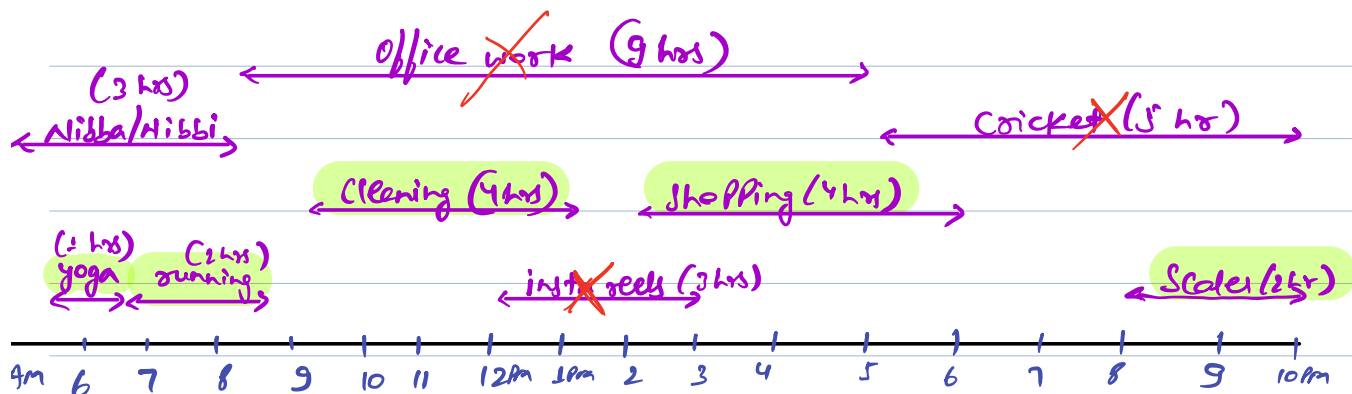


Q) Activity Selection

↳ max Count of task you can do.

Note: ① Start a task, we need to complete.

② At any point of time, single task.



① min duration task first → incorrect

↳ yoga, running, scales, instareels
ans = 4

② ending early

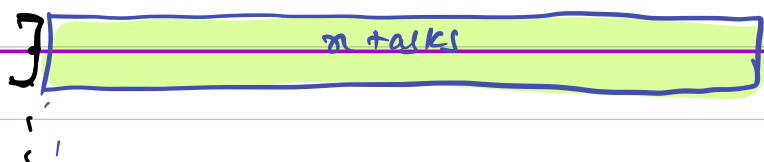
↳ yoga, running, cleaning, shopping, scales
if
ans = 5

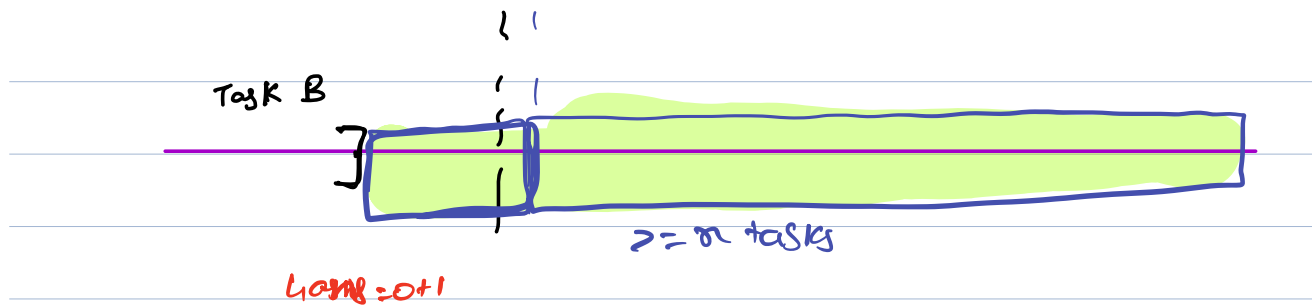
③ starting early → incorrect

↳ nibba/nibbi, office work, cricket
if
ans = 3

Correctness:

Task A





$\rightarrow$ st & et
 Ex: Pair [] arr = $\{(20, 22) (12, 15) (5, 8) (8, 17) (17, 22) (14, 18) (9, 13) (5, 7) (7, 9)\}$

Algorithm:

① Sort all the tasks on the basis of end time.

arr = $(5, 7) (\cancel{8, 8}) (7, 9) (9, 13) (\cancel{12, 15}) (\cancel{8, 17}) (14, 18) (20, 22) (\downarrow i) (17, 22)$

endtime: $7, 8, 13, 18, 22$

ans = $\cancel{1}, 2, 3, 4, 5$

② get all non-overlapping tasks.

T.C: $O(N \log N)$

S.C: $O(1)$

Job Scheduling

- Given n tasks to complete
- Deadline assigned for each task, day on or before we can do task.
- Payment assigned to each task.
- on any given day we can perform only 1 task and Each task take 1 day to finish.
- find max payment we can get.

Ex1:

Job	deadline day	Payment
a	3	100
b	1	19
c	2	27
d	1	25
e	3	30

Parameter
 ↙ ↘
 Payment deadline
 ↘ ↙
 Payment + deadline

a	3	100
b	1	19
c	2	27
d	1	25
e	3	30

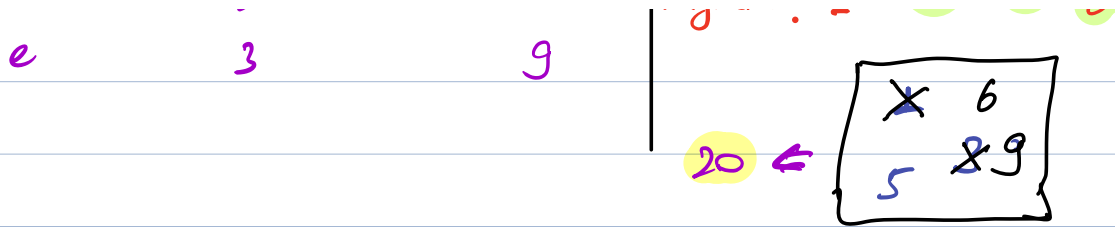
etc - a

Ex2:

Job	deadline day	Payment
a	3	5
b	1	1
c	3	6
d	2	3

Sort jobs on the basis of deadline.

jobs: b d a c e
 deadline: 1 2 3 3 3
 Payment: 1 3 5 6 9



Ex2:

Tasks:	1	2	3	4	5	6	7	8	9	10
dead:	2	1	1	1	4	5	4	5	5	2
money:	250	280	260	350	300	100	260	600	400	150

Step1: Sort on the basis of deadline.

1 task ≤ 1 day

Tasks:	2	3	7	1	10	5	7	6	8	9	↓ i
dead:	1	1	1	2	2	4	4	5	5	5	
money:	250	200	350	200	150	300	250	100	600	400	

task:

4	2	5	7	8
1	2	3	4	5

min heap

~~280~~
600

400
350

~~200~~
250

300
~~100~~

~~250~~
300

600
350

250
~~400~~

⇒ Add all of them

↓

Ans: 1900

(ii) iterate and Pick the jobs if Payment is better using a minHeap.